

Rapport de stage

Méthodes hybrides pour l'ordonnancement sous contraintes de ressources complexes

July 24, 2026

Stagiaire :

Alex TSAN alex.tsan@utoulouse.fr

Tuteurs de stage :

Christian ARTIGUES
Carla JUVIN
Pierre LOPEZ

Mot clés : Recherche opérationnelle; Programmation par contrainte; Décomposition de Benders; Programmation Linéaire

Abstract :

L'optimisation de l'ordonnancement de projets avec ressources multi-compétences et flexibilité d'affectation (MS-RCPSP-AF) pose d'importants défis combinatoires. Ce stage vise à comparer différents paradigmes de résolution exacte pour ce problème. Nous analysons et évaluons les performances de modèles monolithiques en Programmation Linéaire (PLNE) et en Programmation par Contraintes (CP) face à une méthode de décomposition. Nous présentons d'abord une formulation PLNE indexée par le temps et un modèle CP monolithique discret découpant les tâches en sous-intervalles unitaires pour modéliser le remplacement dynamique des opérateurs. Pour le modèle CP, nous étudions également une formulation alternative basée sur la contrainte globale de cardinalité (GCC) et l'ajout de contraintes de bris de symétrie. Face aux limites physiques et combinatoires de ces modèles monolithiques, nous implémentons un algorithme de Décomposition de Benders basée sur la Logique (LBBD). Cette méthode découple l'ordonnancement global, traité par un problème maître en CP, de l'affectation des travailleurs à chaque période, résolue par des algorithmes de flot maximum. Les résultats expérimentaux obtenus sur les benchmarks de la littérature (Set 1a et MSLIB) montrent que la décomposition LBBD résout entre 94 % et 97 % des instances en quelques secondes, surpassant nettement les approches monolithiques.

Contents

Introduction	1
Contexte et Présentation du LAAS-CNRS	1
Énoncé du problème et objectifs du stage	2
Annonce du plan	4
1 Programmation linéaire en nombres entiers	5
1.1 Structure d'un modèle PLNE	5
1.2 Mécanismes de résolution	5
1.3 Application au problème du MS-RCPSP-AF	6
1.3.1 Ensembles	6
1.3.2 Paramètres	6
1.3.3 Variables de décision	6
1.3.4 Fonction objectif	6
1.3.5 Contraintes	6
1.4 Limites et verrous de la formulation PLNE	7
1.4.1 Explosion du nombre de variables avec l'horizon temporel	7
1.4.2 Faiblesse de la relaxation continue	8
1.4.3 Linéarisation des relations logiques	8
2 Programmation par Contraintes	9
2.1 Formalisme des Problèmes de Satisfaction de Contraintes (CSP)	9
2.2 Mécanismes de résolution : Propagation et Recherche	9
2.3 Primitives d'ordonnancement de CP Optimizer	10
2.3.1 Variables d'intervalles (Interval Variables)	10
2.3.2 Contraintes temporelles	10
2.3.3 Ressources disjonctives et cumulatives	10
2.4 Modélisation CP monolithique pour le MS-RCPSP-AF	11
2.4.1 Ensembles et paramètres	11
2.4.2 Variables de décision	11
2.4.3 Formulation mathématique	11
2.4.4 Explication des contraintes	11

2.4.5	Contraintes redondantes de capacité globale	12
2.5	Variante : Modélisation par Contrainte de Cardinalité Globale (GCC)	12
2.5.1	Variables de décision du modèle GCC	12
2.5.2	Formulation par la contrainte Distribute	13
2.5.3	Limites et retour d'expérience sur la formulation GCC	13
2.6	Contraintes de bris de symétrie	14
2.7	Analyse des limites du modèle CP monolithique	14
2.7.1	Explosion dimensionnelle du nombre de variables	14
2.7.2	Perte de la structure globale d'ordonnancement et faiblesse de la propagation	15
2.7.3	Symétries et inefficacité des contraintes de bris de symétrie	15
3	Décomposition de Benders basée sur la Logique	16
3.1	Principes de la Décomposition de Benders basée sur la Logique (LBBD)	16
3.2	Problème Maître d'Ordonnancement	16
3.2.1	Formulation mathématique	16
3.3	Sous-problème d'Affectation	17
3.3.1	Formulation sous forme de Flot Maximum (Max-Flow)	18
3.4	Génération des coupes de Benders	18
3.4.1	Formulation duale et Coupe Minimale (Min-Cut)	19
3.4.2	Formulation de la coupe injectée	19
4	Évaluation Expérimentale	20
4.1	Protocole expérimental et instances de test	20
4.1.1	Environnement de test	20
4.1.2	Jeu de données initial : Set 1a	20
4.1.3	Jeu de données de passage à l'échelle : MSLIB	20
4.2	Comparaison globale des approches sur le Set 1a	21
4.2.1	Analyse des taux de résolution et des temps de calcul	22
4.2.2	Comparaison directe sur les instances faciles (communes)	22
4.3	Analyse des performances et discussion	22
4.3.1	Le temps de convergence de Benders	22
4.3.2	Analyse de l'impact des contraintes de bris de symétrie	22
4.4	Visualisations graphiques pour le Set 1a	22

4.5	Résultats sur le jeu de données de passage à l'échelle MSLIB1 (Subset 1)	23
4.5.1	Analyse des performances sur MSLIB1	23
Conclusion et Perspectives		25

Introduction

Contexte et Présentation du LAAS-CNRS

Ce document présente les travaux de stage de fin d'année de Master 1 Informatique, parcours Intelligence Artificielle Fondements et Applications (IAFA) à l'Université de Toulouse. Ce stage s'est déroulé au LAAS-CNRS (Laboratoire d'Analyse et d'Architecture des Systèmes) à Toulouse. Fondé en 1968, le LAAS est une Unité Propre de Recherche (UPR 8001) du CNRS, rattachée à l'Institut des sciences de l'information et de leurs interactions (IS2I) et à l'Institut des sciences de l'ingénierie et des systèmes (INSIS). Le laboratoire est conventionné avec les principaux établissements d'enseignement supérieur de la région : l'Université de Toulouse, l'INSA Toulouse, l'INP Toulouse et l'ISAE-SUPAERO. Il regroupe environ 650 personnes, comprenant des chercheurs, enseignants-chercheurs, ingénieurs, techniciens, doctorants et post-doctorants.

Les activités du LAAS-CNRS s'organisent autour de quatre thèmes principaux : l'Automatique, l'Informatique, les Micro et Nano Systèmes, et la Robotique. Ce spectre permet de lier le développement matériel aux algorithmes de décision.

La recherche au LAAS-CNRS s'articule autour de six départements : DO, GE, HOPES, MNBT, RISC et ROB.

- **Décisions et Optimisation** : Le département DO développe des théories mathématiques et des outils algorithmiques pour la prise de décision et la commande de systèmes complexes. Ses recherches s'articulent autour de deux approches complémentaires : la synthèse de lois de décision via des techniques d'optimisation et la résolution de problèmes d'optimisation combinatoire discrets. Ses travaux trouvent des applications dans les réseaux électriques, les transports et l'industrie du futur. C'est dans ce cadre, axé sur l'optimisation combinatoire sous contraintes, que s'intègrent les recherches de l'équipe ROC en ordonnancement et planification.
- **Gestion de l'Energie** : Le département GE se consacre à l'optimisation de l'efficacité énergétique, à la robustesse et à la fiabilité des architectures de conversion de puissance. Pour répondre aux défis de la transition écologique, de la décentralisation des énergies renouvelables et de l'électrification des transports, il conçoit des composants électroniques miniatures et des systèmes de gestion de l'énergie. Le département valide ses prototypes au sein du bâtiment expérimental connecté ADREAM.
- **Systèmes Hyperfréquences et Photoniques** : Le département HOPES explore la convergence entre l'électronique de pointe et la photonique pour concevoir les briques de base de l'Internet des objets (IoT) et des futurs systèmes cyber-physiques. Ses recherches visent à créer des composants miniatures et à faible consommation d'énergie, capables de communiquer avec leur environnement.
- **Micro Nano Bio Technologies** : Le département MNBT combine des approches fondamentales et appliquées pour concevoir des nanomatériaux et des nano-objets, puis les intégrer dans des dispositifs complexes. Ses activités ciblent trois défis : les technologies pour le numérique, les technologies pour la santé et la surveillance environnementale.
- **Réseaux Informatique Systèmes de Confiance** : Le département RISC cible la maîtrise des architectures informatiques et des réseaux de communication distribués. Ses recherches se focalisent sur la cybersécurité, l'orchestration dynamique des ressources cloud et le développement d'une intelligence artificielle digne de confiance et frugale en énergie.
- **Robotique** : Le département ROB déploie une recherche dédiée à l'autonomie motrice et cognitive des systèmes physiques opérant dans des environnements réels et non maîtrisés. Ses

domaines d'expertise couvrent la planification de mouvements pour les structures anthropomorphes, le couplage entre la perception et l'action, ainsi que la gestion des interactions entre le robot, l'Homme et son milieu.

Ce stage s'est déroulé au sein du département Décisions et Optimisation (DO), dans l'équipe ROC (Recherche Opérationnelle, Combinatoire et Contraintes) dirigée par Christian Artigues. L'équipe ROC se consacre à la résolution de problèmes d'optimisation combinatoire discrets complexes. Son objectif est de concevoir des modèles mathématiques et des algorithmes pour trouver les meilleures décisions possibles face à un grand nombre de choix. Ses recherches s'organisent autour de trois piliers méthodologiques :

- **La Programmation Linéaire** : Formulations mathématiques et approches polyédrales.
- **La Programmation par Contraintes** : Exploitation de structures logiques et de relations temporelles (variables d'intervalles).
- **Les Méthodes Hybrides et Décompositions** : Couplage de plusieurs paradigmes (comme la décomposition de Benders logique) pour traiter les problèmes de grande taille.

Énoncé du problème et objectifs du stage

Le problème abordé s'inscrit dans le domaine de l'optimisation combinatoire et de l'ordonnancement sous contraintes de ressources. Plus précisément, il s'agit du problème d'ordonnancement de projet sous contraintes de ressources multi-compétences avec flexibilité d'affectation (Multi-Skill Resource-Constrained Project Scheduling Problem with Allocation Flexibility, ou MS-RCPSP-AF).

Dans sa formulation classique (RCPSP), l'ordonnancement consiste à planifier des tâches liées par des précédences tout en respectant la capacité limitée de ressources cumulatives (ressources disponibles en quantité globale constante à chaque instant, comme un parc de machines), avec pour objectif de minimiser la durée totale du projet (makespan). La variante MS-RCPSP (MS pour Multi-Skill) gère des ressources humaines avec des profils hétérogènes : les opérateurs maîtrisent une ou plusieurs compétences spécifiques. Les besoins d'une tâche sont définis par des exigences en compétences. Dans ce cadre traditionnel, l'affectation est rigide, ce qui implique que les opérateurs sélectionnés restent sur une tâche du début à la fin.

L'introduction de la flexibilité d'affectation (MS-RCPSP-AF), concept introduit par Juvin et al. [1], lève cette hypothèse en autorisant la rotation ou le remplacement des opérateurs au cours de l'exécution d'une tâche. La seule condition requise est que les exigences en compétences de la tâche soient couvertes à chaque période.

Pour illustrer l'intérêt de cette flexibilité d'allocation (AF), la Figure 1 compare deux scénarios d'exécution d'un projet composé de trois tâches (Tâche A de durée 10 requérant s_1 , Tâche B de durée 5 requérant s_2 , et Tâche C de durée 5 requérant s_3) à réaliser par deux travailleurs (W_1 maîtrisant $\{s_1, s_2\}$ et W_2 maîtrisant $\{s_1, s_3\}$) :

- **Scénario 1 (Sans Flexibilité)** : L'affectation est rigide. Si W_1 commence la Tâche B à l'instant 0, il y reste jusqu'à sa fin à l'instant 5. La Tâche A ne peut pas démarrer en parallèle car W_2 ne peut pas être affecté à la Tâche A du début à la fin sous peine de ne pas pouvoir réaliser la Tâche C (exigeant la compétence exclusive s_3 de W_2) à l'instant 5. La Tâche A est décalée et ne commence qu'à l'instant 5, affectée à W_1 , pour un Makespan de 15 heures.
- **Scénario 2 (Avec Flexibilité)** : Un remplacement est possible. La Tâche A débute dès l'instant 0, prise en charge par W_2 pour sa première moitié. À l'instant 5, la Tâche C doit

démarrer et requiert W_2 pour sa compétence s_3 . Grâce à la flexibilité, W_2 cède sa place sur la Tâche A à W_1 (qui vient de terminer la Tâche B). La Tâche A se poursuit sans interruption et se termine à l'instant 10, tout comme la Tâche C. Le Makespan est ainsi réduit de 15h à 10h.

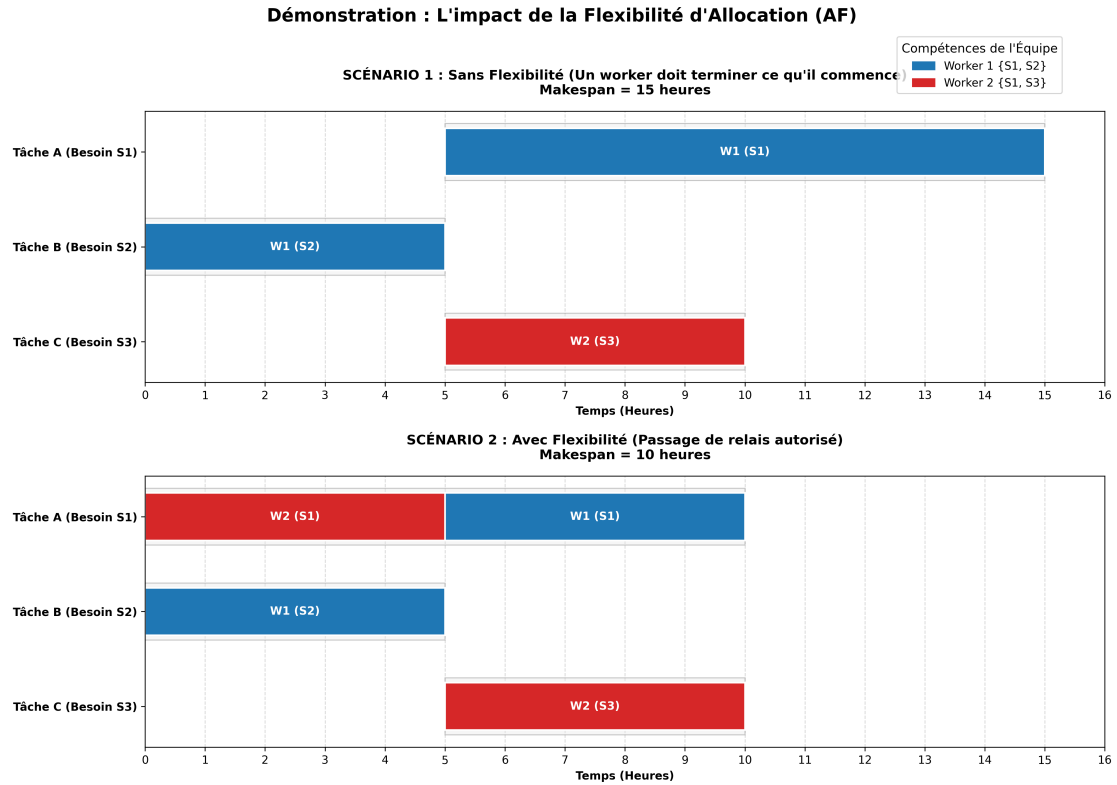


Figure 1: Illustration de l'impact de la flexibilité d'affectation (AF) sur le Makespan d'un projet

Si cette flexibilité permet d'optimiser l'usage des ressources et de réduire le makespan, elle augmente la complexité du problème. Contrairement aux approches classiques où l'affectation est fixe pour toute la durée d'une tâche, la flexibilité impose de valider la faisabilité de l'affectation à chaque unité de temps. Cela génère une combinatoire importante, renforcée par des symétries d'affectation lorsque les opérateurs partagent des compétences identiques. Les solveurs classiques peinent alors à élaguer l'arbre de recherche.

La littérature de la recherche opérationnelle s'est intéressée au problème classique d'ordonnancement de projets (RCPSP). L'intégration de ressources multi-compétences, introduite par Bellenguez-Morineau et Néron [2, 3], a suscité un intérêt croissant, comme le montre la revue de littérature de Snauwaert et Vincke [4]. Les approches exactes traditionnelles reposent souvent sur la programmation linéaire en nombres entiers (MILP), étudiée par Almeida et al. [5], mais ces formulations présentent des limites de passage à l'échelle lorsque l'horizon s'allonge. Pour y remédier, des approches basées sur la génération de colonnes (Branch-and-Price) ont été explorées [6, 7]. Parallèlement, la Programmation par Contraintes (PPC) s'est imposée comme une alternative pour modéliser des contraintes complexes [8, 9]. Néanmoins, ces modèles monolithiques atteignent leurs limites sur des instances de grande taille en raison de la combinatoire induite par la flexibilité. C'est pourquoi la recherche s'oriente vers des méthodes de décomposition hybrides. La Décomposition de Benders basée sur la Logique (LBBD), introduite par Hooker [10, 11] et analysée par Cire et al. [12], apparaît comme l'un des frameworks adaptés. Contrairement aux schémas de Benders classiques qui délèguent l'ordonnancement au sous-problème, les approches d'ordonnancement multi-compétences inversent ce schéma en résolvant l'ordonnancement global dans le problème maître en

CP et l'affectation dans le sous-problème. Cette inversion, introduite par Yi-fan et al. [13] pour le MSPSP classique, a été étendue par Juvin et al. [1] pour modéliser la flexibilité d'affectation (AF) dans le cadre du MS-RCPSP-AF, permettant le remplacement des travailleurs au cours de l'exécution des tâches.

Annonce du plan

Ce manuscrit est structuré en quatre chapitres principaux :

Le Chapitre 1 introduit la modélisation mathématique du problème sous forme de programme linéaire en nombres entiers (PLNE). Cette approche permet de formaliser les contraintes d'affectation et de précédence, tout en mettant en évidence les limites inhérentes à ce type de formulation.

Le Chapitre 2 explore la modélisation en Programmation par Contraintes (CP). Nous présentons la formulation classique par sous-intervalles unitaires, une variante reposant sur la contrainte de cardinalité globale (GCC) et des contraintes de bris de symétrie, en analysant leurs limites calculatoires respectives.

Le Chapitre 3 présente la Décomposition de Benders basée sur la Logique (LBBD). Ce chapitre détaille la génération de coupes logiques et le couplage entre un Problème Maître en CP pour l'ordonnancement global et un Sous-Problème d'affectation résolu par flot maximum.

Enfin, le Chapitre 4 présente l'évaluation expérimentale des approches sur des instances de référence (Set 1a) et de passage à l'échelle (MSLIB), confirmant la supériorité de la décomposition face à l'explosion combinatoire.

1 Programmation linéaire en nombres entiers

La Programmation Linéaire en Nombres Entiers (PLNE, ou ILP en anglais) est un outil d'optimisation classique en Recherche Opérationnelle pour résoudre les problèmes de décision complexes. La PLNE permet de modéliser des problèmes NP-difficiles [14] en combinant une formulation mathématique avec la puissance des solveurs numériques [15].

Dans le cadre du problème d'ordonnancement de projets sous contraintes de ressources (RCPSP) et de ses extensions multi-compétences (MS-RCPSP), la PLNE a constitué l'une des premières approches de résolution exacte [16]. Les formulations linéaires décrivent les relations temporelles entre tâches et les contraintes d'affectation. En particulier, les formulations indexées par le temps (où l'horizon temporel est discrétisé en unités de temps) servent de référence dans la littérature pour évaluer d'autres paradigmes [5]. Ces modèles linéaires sont aussi utilisés dans des approches hybrides, que ce soit pour le problème maître de Benders [17, 18] ou pour la génération de colonnes [6, 7].

Ce chapitre présente les bases théoriques de la PLNE, les mécanismes de résolution des solveurs, puis détaille la modélisation mathématique du problème du MS-RCPSP-AF inspirée de Polo-Mejía et al. [9, 19, 20].

1.1 Structure d'un modèle PLNE

Un programme linéaire en nombres entiers comporte trois éléments principaux :

Les variables de décision : Ce sont les inconnues à déterminer pour résoudre le problème. En PLNE, ces variables prennent des valeurs entières, et sont souvent binaires (0 ou 1) pour représenter des choix de type oui/non.

La fonction objectif : Une fonction linéaire à optimiser (généralement la minimisation de la date de fin de projet, le Makespan). Elle s'exprime comme une somme de variables pondérées par des coefficients.

Les contraintes : Un ensemble d'équations et d'inégalités linéaires qui limitent l'espace des solutions réalisables (capacités, précédences ou limites de ressources). Les multiplications de variables ou les conditions logiques directes y sont interdites pour conserver la linéarité.

1.2 Mécanismes de résolution

Contrairement à la programmation linéaire continue, qui se résout en temps polynomial à l'aide de l'algorithme du Simplexe, l'intégralité des variables rend la PLNE NP-difficile. L'espace des solutions réalisables se limite à un ensemble de points discrets.

Pour explorer cet espace, les solveurs modernes (comme Gurobi ou IBM ILOG CPLEX) combinent la programmation linéaire continue avec des algorithmes d'exploration arborescente, principalement la méthode de Séparation et Évaluation (Branch-and-Bound) et le Branch-and-Cut.

La relaxation continue : À chaque nœud de l'arbre de recherche, le solveur résout le problème en omettant la contrainte d'intégralité des variables, ce qui fournit une borne inférieure pour un problème de minimisation.

Branching : Si une variable devant être entière prend une valeur fractionnaire (par exemple 2,4) dans la relaxation continue, le solveur sépare l'espace de recherche en créant deux sous-nœuds : l'un imposant que la variable soit inférieure ou égale à 2, et l'autre supérieure ou égale à 3.

L'élagage (Bounding and Pruning) : L'arbre est exploré de manière itérative. Si la borne continue calculée à un nœud est moins bonne qu'une solution entière déjà connue (la borne supérieure), le solveur coupe ce nœud et sa descendance pour éviter une recherche inutile.

1.3 Application au problème du MS-RCPSP-AF

Nous formalisons ici le problème du MS-RCPSP-AF sous forme de modèle PLNE pour poser les bases mathématiques du problème avant de détailler la modélisation par Programmation par Contraintes.

1.3.1 Ensembles

- $\mathcal{A}$: Ensemble des activités à planifier.
- $\mathcal{L}$: Ensemble des différentes compétences.
- $\mathcal{W}$: Ensemble des ressources multi-compétences (travailleurs).
- $\mathcal{CR}$: Ensemble des ressources cumulatives.
- $\mathcal{T}$: Horizon temporel, $\mathcal{T} = \{1, \dots, T\}$, indexé par t .
- E : Ensemble des contraintes de précédence.

1.3.2 Paramètres

- p_i : Temps de traitement de l'activité i .
- B_k : Quantité de ressource k .
- $b_{i,k}$: Quantité de ressource k requise pour l'activité i .
- $a_{i,l}$: Nombre de travailleurs maîtrisant la compétence l requis pour l'activité i .
- $m_{w,l}$: Égal à 1 si le travailleur w maîtrise la compétence l , 0 sinon.
- q_i : Nombre minimum de travailleurs requis pour l'activité i .
- r_i, d_i : Date de disponibilité et date d'échéance autorisées pour l'activité i .

1.3.3 Variables de décision

- $x_{i,t} \in \{0, 1\}$: Égal à 1 si l'activité i est exécutée à l'instant t , 0 sinon.
- $z_{w,i,l,t} \in \{0, 1\}$: Égal à 1 si le travailleur w est affecté à l'activité i pour exercer la compétence l à l'instant t , 0 sinon.

1.3.4 Fonction objectif

L'objectif est de minimiser la durée totale d'exécution du projet (*Makespan*) :

$$\min C_{\max} \quad (1)$$

1.3.5 Contraintes

Unicité d'affectation : Un travailleur peut être affecté au maximum à une seule activité et une seule compétence à l'instant t .

$$\sum_{i \in \mathcal{A}} \sum_{l \in \mathcal{L}} z_{w,i,l,t} \leq 1 \quad \forall w \in \mathcal{W}, \forall t \in \mathcal{T} \quad (2)$$

Validation de la maîtrise des compétences : Un travailleur ne peut être affecté à une compétence que s'il la maîtrise.

$$z_{w,i,l,t} \leq m_{w,l} \quad \forall w \in \mathcal{W}, \forall i \in \mathcal{A}, \forall l \in \mathcal{L}, \forall t \in \mathcal{T} \quad (3)$$

Couverture des exigences en compétences : Si une activité est active ($x_{i,t} = 1$), le nombre d'opérateurs affectés à la compétence l doit couvrir le besoin requis $a_{i,l}$.

$$\sum_{w \in \mathcal{W}} z_{w,i,l,t} \geq a_{i,l} \cdot x_{i,t} \quad \forall l \in \mathcal{L}, \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (4)$$

Respect de l'effectif minimum requis pour une activité : Le nombre total d'affectations pour l'activité i à l'instant t doit être supérieur ou égal à l'exigence q_i .

$$\sum_{w \in \mathcal{W}} \sum_{l \in \mathcal{L}} z_{w,i,l,t} \geq q_i \cdot x_{i,t} \quad \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (5)$$

Respect des durées de traitement : Chaque activité doit s'exécuter pendant exactement p_i périodes sur son intervalle de temps admissible.

$$\sum_{t=r_i}^{d_i} x_{i,t} = p_i \quad \forall i \in \mathcal{A} \quad (6)$$

Contraintes de précédence : Une activité successeur j ne peut pas démarrer tant que son activité prédécesseur i n'est pas achevée.

$$p_i \cdot (1 - x_{j,t}) \geq \sum_{t'=t}^T x_{i,t'} \quad \forall (i, j) \in E, \forall t \in \mathcal{T} \quad (7)$$

Capacité des ressources matérielles cumulatives : L'utilisation des ressources matérielles cumulatives à l'instant t est limitée à leur capacité globale B_k .

$$\sum_{i \in \mathcal{A}} b_{i,k} \cdot x_{i,t} \leq B_k \quad \forall k \in \mathcal{CR}, \forall t \in \mathcal{T} \quad (8)$$

Définition de la borne inférieure du Makespan

$$C_{\max} \geq t \cdot x_{i,t} \quad \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (9)$$

Cette contrainte lie la variable $C_{\max}$ au déroulement du projet. Le modèle étant indexé par le temps, si une activité i s'exécute à l'instant t ($x_{i,t} = 1$), l'inéquation force le $C_{\max}$ à être supérieur ou égal à cet instant t . Lors de la minimisation de la fonction objectif, la variable $C_{\max}$ prend la valeur du dernier instant t où une tâche est active, ce qui correspond à la date de fin globale.

1.4 Limites et verrous de la formulation PLNE

Le modèle PLNE fournit une formulation exacte du problème du MS-RCPSP-AF, mais présente des limites pratiques pour la résolution d'instances de taille moyenne ou grande.

1.4.1 Explosion du nombre de variables avec l'horizon temporel

L'utilisation de variables indexées par le temps ($x_{i,t}$ et $z_{w,i,l,t}$) rend la taille du modèle dépendante de l'horizon temporel T . Le nombre de variables d'affectation $z_{w,i,l,t}$ s'élève à $|\mathcal{W}| \times |\mathcal{A}| \times |\mathcal{L}| \times T$.

Si T est grand, le solveur doit gérer un grand nombre de variables binaires, ce qui sature la mémoire et rend la phase de séparation (*Branching*) longue et difficile.

1.4.2 Faiblesse de la relaxation continue

Les formulations indexées par le temps pour l'ordonnancement présentent souvent une relaxation continue faible. Lorsque les variables binaires $x_{i,t}$ et $z_{w,i,l,t}$ sont relâchées dans l'intervalle $[0, 1]$, le solveur linéaire trouve des solutions fractionnaires où les tâches sont fragmentées dans le temps. La borne inférieure fournie par cette relaxation est alors éloignée de la valeur entière optimale, ce qui oblige le solveur à explorer un grand nombre de nœuds dans l'arbre de recherche.

1.4.3 Linéarisation des relations logiques

Conserver un formalisme linéaire impose de linéariser les contraintes logiques. Pour les contraintes de précédence (7) :

$$p_i \cdot (1 - x_{j,t}) \geq \sum_{t'=t}^T x_{i,t'} \quad \forall (i, j) \in E, \forall t \in \mathcal{T} \quad (10)$$

Si le successeur j s'exécute à l'instant t ($x_{j,t} = 1$), le membre de gauche s'annule, forçant le prédécesseur i à être inactif à partir de cet instant t . Si j n'est pas actif à l'instant t ($x_{j,t} = 0$), la contrainte devient triviale ($p_i \geq \sum x_{i,t'}$) car la somme des périodes d'activité de i ne dépasse pas sa durée p_i . Cette linéarisation doit être écrite pour chaque période t et chaque arc du graphe de précédence E , ce qui augmente le nombre de contraintes à gérer par le solveur.

2 Programmation par Contraintes

La Programmation par Contraintes (PPC, ou CP en anglais) est un paradigme d'optimisation exacte adapté à la résolution de problèmes combinatoires comme la planification et l'ordonnancement. Contrairement à la programmation linéaire, ce paradigme permet d'exprimer des contraintes non linéaires, logiques ou globales.

En CP, l'utilisateur formule le problème sous forme de contraintes, et le processus de recherche de solutions et de preuve d'optimalité est pris en charge par un solveur générique. Ce solveur repose sur le couplage entre la propagation de contraintes et l'exploration arborescente. Les contraintes globales encapsulent des structures combinatoires récurrentes (telles que l'ordonnancement de tâches ou l'affectation), et leur sont associés des algorithmes de filtrage (propagateurs). Pour l'ordonnancement sous contraintes cumulatives, ces propagateurs utilisent des algorithmes comme le *timetabling* [21], le *cumulative edge-finding* [22] ou le raisonnement énergétique [23].

L'application de la CP au problème de l'ordonnancement de projet multi-compétences (MSPSP) a donné lieu à des travaux notables. Des approches basées sur le CP/SAT, le *lazy clause generation* et l'apprentissage de no-goods ont été développées par Young et al. [8]. Pour le MS-RCPSP avec préemption et contraintes de ressources, Polo-Mejía et al. [9, 19] ont proposé des modèles en CP exploitant les variables d'intervalles de solveurs comme *IBM ILOG CP Optimizer*. Par ailleurs, des travaux comme ceux de Carlier et Néron [24] ou de Baptiste et Bonifas [25] ont montré l'intérêt d'ajouter des enveloppes cumulatives redondantes pour accélérer la convergence.

Ce chapitre présente le formalisme général des CSP, décrit les primitives d'ordonnancement de CP Optimizer, formule le modèle CP monolithique pour le MS-RCPSP-AF, puis analyse ses limites combinatoires.

2.1 Formalisme des Problèmes de Satisfaction de Contraintes (CSP)

Un CSP est défini par un triplet (X, D, C) où :

- $X = \{x_1, x_2, \dots, x_n\}$ est l'ensemble des variables de décision.
- $D = \{D(x_1), D(x_2), \dots, D(x_n)\}$ représente l'ensemble des domaines associés (les valeurs admissibles pour chaque variable, généralement des entiers).
- $C = \{c_1, c_2, \dots, c_m\}$ est l'ensemble des contraintes qui limitent les combinaisons de valeurs possibles pour les variables.

L'objectif est d'affecter à chaque variable x_i une valeur de son domaine $D(x_i)$ en satisfaisant toutes les contraintes de C . Dans le cas d'un problème d'optimisation (COP), on cherche à minimiser ou maximiser une fonction objectif $f(X)$.

2.2 Mécanismes de résolution : Propagation et Recherche

Pour explorer l'espace des solutions, les solveurs de CP (comme *IBM ILOG CP Optimizer*) couplent deux mécanismes :

1. **La propagation de contraintes (filtrage) :** À chaque contrainte $c_j \in C$ est associé un propagateur. Lorsqu'une décision restreint le domaine d'une variable, le propagateur évalue la cohérence locale (comme l'arc-cohérence) et élimine des domaines des autres variables connectées les valeurs devenues incompatibles.
2. **La recherche arborescente (exploration) :** Si la propagation ne suffit pas à obtenir une solution unique, le solveur effectue un branchement (*branching*) : il choisit une variable à domaine multiple, la fixe à une valeur, puis relance la propagation. Si une contrainte est violée (un domaine devient vide), le solveur effectue un retour arrière (*backtracking*) pour essayer une autre branche.

2.3 Primitives d'ordonnement de CP Optimizer

Pour modéliser l'ordonnement, les solveurs comme *IBM ILOG CP Optimizer* disposent de primitives de haut niveau qui évitent de discrétiser le temps de façon externe.

2.3.1 Variables d'intervalles (Interval Variables)

Une variable d'intervalle, notée a , représente une tâche ou une partie de tâche définie par un début $a.start$, une fin $a.end$, et une durée $a.size$, liés par la relation $a.end = a.start + a.size$. De plus, une variable d'intervalle peut être **optionnelle** : le solveur décide alors de sa présence ou non dans la solution ($a.present \in \{0,1\}$). Si l'intervalle est absent, ses attributs de temps sont ignorés par les contraintes associées.

2.3.2 Contraintes temporelles

Les relations de précédence et de synchronisation entre activités s'expriment avec des contraintes spécifiques :

- **endBeforeStart(a, b, z)** : Impose que le début de l'intervalle b soit postérieur ou égal à la fin de l'intervalle a , avec un délai de transition z :

$$b.start \geq a.end + z \quad (11)$$

Cette contrainte n'est active que si a et b sont présents.

- **startAtEnd(a, b)** : Impose que l'intervalle a commence exactement au moment où l'intervalle b se termine :

$$a.start = b.end \quad (12)$$

Cette contrainte sert à modéliser des jonctions temporelles sans interruption.

- **span($a, \{b_1, \dots, b_k\}$)** : Force l'intervalle a à couvrir l'ensemble des intervalles présents du groupe $\{b_1, \dots, b_k\}$. Ainsi, a débute au début du premier intervalle actif et se termine à la fin du dernier intervalle actif du groupe :

$$a.start = \min_{1 \leq i \leq k} \{b_i.start \mid b_i.present = 1\} \quad (13)$$

$$a.end = \max_{1 \leq i \leq k} \{b_i.end \mid b_i.present = 1\} \quad (14)$$

- **alternative($a, \{b_1, \dots, b_k\}, c$)** : Associe un intervalle global a à un ensemble de sous-intervalles optionnels $\{b_1, \dots, b_k\}$. Si a est présent, exactement c sous-intervalles parmi $\{b_1, \dots, b_k\}$ doivent être présents, et ces c sous-intervalles sélectionnés doivent commencer et finir en même temps que a . Dans sa version standard ($c = 1$), cette contrainte modélise le choix d'une ressource unique parmi plusieurs alternatives.

2.3.3 Ressources disjonctives et cumulatives

- **Non-chevauchement (noOverlap)** : Pour modéliser une ressource disjonctive (comme un travailleur qui ne peut exécuter qu'une seule tâche à la fois), on regroupe les variables d'intervalles affectées à cette ressource dans une contrainte **noOverlap**. Elle garantit que deux intervalles présents ne se chevauchent pas dans le temps.

2.4 Modélisation CP monolithique pour le MS-RCPSP-AF

Afin de modéliser la flexibilité d'affectation (rotation des équipes) sans préemption de la tâche globale, nous développons un modèle par Programmation par Contraintes fondé sur la discrétisation interne des activités en parties unitaires.

2.4.1 Ensembles et paramètres

Le modèle réutilise les ensembles de base introduits au Chapitre 1 ($\mathcal{A}, \mathcal{L}, \mathcal{W}, \mathcal{CR}, E$) et y ajoute l'ensemble suivant :

- $V_i = \{1, \dots, p_i\}$: Ensemble des parties de durée unitaire de l'activité $i \in \mathcal{A}$, indexé par v .

Nous définissons le paramètre supplémentaire suivant :

- N_l : Nombre total de travailleurs maîtrisant la compétence l dans l'ensemble $\mathcal{W}$.

2.4.2 Variables de décision

Le modèle utilise des variables d'intervalles à plusieurs niveaux :

- $C_{\max} \geq 0$: Variable entière représentant la date de fin globale du projet.
- act_i : Variable d'intervalle représentant l'exécution globale de l'activité i , de taille p_i .
- $par_{i,v}$: Variable d'intervalle de durée unitaire représentant la v -ème partie de l'activité i (pour $v \in V_i$).
- $awo_{w,i,l,v}$: Variable d'intervalle optionnelle de durée unitaire représentant l'affectation du travailleur w exerçant la compétence l pour la v -ème partie de l'activité i .

2.4.3 Formulation mathématique

Le modèle CP monolithique s'écrit de la manière suivante :

$$\min \quad C_{\max} \tag{15}$$

s. t.

$$C_{\max} \geq act_i.end \quad \forall i \in \mathcal{A} \tag{16}$$

$$span(act_i, \{par_{i,v} \mid v \in V_i\}) \quad \forall i \in \mathcal{A} \tag{17}$$

$$startAtEnd(par_{i,v+1}, par_{i,v}) \quad \forall i \in \mathcal{A}, \forall v \in \{1, \dots, p_i - 1\} \tag{18}$$

$$endBeforeStart(act_i, act_m) \quad \forall (i, m) \in E \tag{19}$$

$$\sum_{i \in \mathcal{A}} pulse(act_i, b_{i,k}) \leq B_k \quad \forall k \in \mathcal{CR} \tag{20}$$

$$noOverlap(\{awo_{w,i,l,v} \mid \forall i \in \mathcal{A}, \forall l \in \mathcal{L}, \forall v \in V_i\}) \quad \forall w \in \mathcal{W} \tag{21}$$

$$alternative(par_{i,v}, \{awo_{w,i,l,v} \mid \forall w \in \mathcal{W} \text{ s.t. } m_{w,l} = 1\}, a_{i,l}) \quad \forall i \in \mathcal{A}, \forall v \in V_i, \forall l \in \mathcal{L} \tag{22}$$

$$alternative(par_{i,v}, \{awo_{w,i,l,v} \mid \forall w \in \mathcal{W}, \forall l \in \mathcal{L}\}, q_i) \quad \forall i \in \mathcal{A}, \forall v \in V_i \tag{23}$$

2.4.4 Explication des contraintes

- L'équation (16) définit le makespan à minimiser comme supérieur ou égal à la fin de chaque activité globale act_i .
- La contrainte (17) lie l'activité globale act_i à l'ensemble de ses sous-intervalles unitaires $par_{i,v}$.

- L'équation (18) assure la contiguïté temporelle (non-préemption) en forçant la fin de la partie v à coïncider avec le début de la partie $v + 1$.
- La contrainte de précédence (19) assure les relations d'ordre du projet entre les activités globales.
- L'équation (20) restreint l'usage simultané des ressources matérielles cumulatives à leur capacité B_k .
- L'équation (21) garantit la disjonction de chaque travailleur : un opérateur w ne peut pas travailler sur plusieurs tâches ou plusieurs compétences en même temps.
- La contrainte alternative (22) modélise la flexibilité : pour chaque partie unitaire v de l'activité i et pour chaque compétence l , elle force la sélection d'exactly $a_{i,l}$ intervalles optionnels d'affectation parmi les travailleurs qualifiés.
- L'équation (23) fait de même pour satisfaire la contrainte d'effectif physique minimum global q_i sur chaque partie unitaire.

2.4.5 Contraintes redondantes de capacité globale

Afin d'accélérer la propagation et d'aider le solveur à détecter plus rapidement les surcharges de capacité de main-d'œuvre, nous introduisons des enveloppes cumulatives globales redondantes :

$$wU = \sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, q_i) \leq |\mathcal{W}| \quad (24)$$

$$sU_l = \sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, a_{i,l}) \leq N_l \quad \forall l \in \mathcal{L} \quad (25)$$

Ces enveloppes ne modifient pas l'espace des solutions, mais renforcent les mécanismes de filtrage en évaluant la demande de main-d'œuvre au niveau des activités globales act_i plutôt que sur chaque partie unitaire.

2.5 Variante : Modélisation par Contrainte de Cardinalité Globale (GCC)

Pour tenter de réduire le nombre de variables d'intervalles optionnelles et de simplifier l'expression de l'affectation, nous avons conçu une modélisation alternative reposant sur la ****Contrainte de Cardinalité Globale**** (GCC, ou *Global Cardinality Constraint*). En CP, la GCC permet d'imposer des bornes (ou des valeurs exactes) sur le nombre d'occurrences de valeurs au sein d'un tableau de variables. Dans CP Optimizer, cette contrainte est disponible via la primitive **distribute**.

L'idée de cette formulation est de remplacer l'affectation par couples tâche-travailleur-compétence (les variables optionnelles $awo_{w,i,l,v}$) par une décision d'affectation centrée sur le travailleur au moyen de variables entières.

2.5.1 Variables de décision du modèle GCC

Pour chaque tâche $i \in \mathcal{A}$, chaque partie unitaire $v \in V_i$, et chaque travailleur $w \in \mathcal{W}$, nous définissons :

- Une variable entière d'affectation de compétence, notée $SW_{w,i,v}$, représentant la compétence que le travailleur w utilise pour réaliser la partie v de la tâche i .
- Le domaine de cette variable $SW_{w,i,v}$ est restreint à :

$$\mathcal{D}(SW_{w,i,v}) = \{0\} \cup (\mathcal{MS}_w \cap \mathcal{SA}_i) \quad (26)$$

où :

- La valeur 0 signifie que le travailleur w n'est pas affecté à la tâche i à l'instant v .
- Un entier $l \geq 1$ représente l'indice de la compétence exercée.
- $\mathcal{MS}_w$ est l'ensemble des compétences maîtrisées par le travailleur w .
- $\mathcal{SA}_i$ est l'ensemble des compétences exigées par la tâche i .
- Une variable d'intervalle optionnelle $AW_{w,i,v}$ représentant la présence du travailleur w sur la partie v de la tâche i . Cette variable d'intervalle est synchronisée avec la variable entière $SW_{w,i,v}$ par la contrainte logique :

$$\text{presence_of}(AW_{w,i,v}) \iff (SW_{w,i,v} > 0) \quad (27)$$

2.5.2 Formulation par la contrainte Distribute

L'exigence en compétences pour chaque tâche est modélisée par une contrainte globale de cardinalité sur chaque intervalle de temps. Pour chaque tâche $i \in \mathcal{A}$ ($p_i > 0$) et chaque partie unitaire $v \in V_i$, nous imposons :

$$\text{distribute}(\text{cards}, \{SW_{w,i,v} \mid w \in \mathcal{W}\}, \text{values}) \quad (28)$$

où :

- $\text{values} = [0, 1, 2, \dots, |L|]$ représente l'ensemble des valeurs d'affectation possibles.
- cards est le vecteur des cardinalités défini par :
 - Pour la valeur 0 (inactivité), le nombre de travailleurs inactifs sur la tâche i doit être égal à :
$$\text{cards}[0] = |\mathcal{W}| - q_i \quad (29)$$
 - Pour chaque compétence $l \in \{1, \dots, |L|\}$, le nombre d'occurrences de la valeur l doit être égal à l'exigence de la tâche :
$$\text{cards}[l] = \text{sreq}_{i,l} \quad (30)$$

Cette contrainte garantit que la demande en compétences de la tâche est satisfaite à chaque unité de temps, tout en s'assurant qu'aucun travailleur non qualifié ne soit affecté à une compétence qu'il ne maîtrise pas (par la restriction des domaines).

2.5.3 Limites et retour d'expérience sur la formulation GCC

Bien que cette formulation théorique réduise le nombre de variables d'intervalles en fusionnant l'affectation et les compétences au sein d'une même variable entière, l'évaluation expérimentale (présentée au chapitre 4 sous le nom **CP Distribute**) a révélé des performances en retrait par rapport au modèle **CP Classique** :

- **Faiblesse de la propagation duale** : Dans CP Optimizer, les variables d'intervalles optionnelles couplées à des contraintes d'alternatives disposent de propagateurs spécialisés robustes. Le passage par des variables entières intermédiaires via la primitive **distribute** affaiblit le niveau de cohérence locale.
- **Perte d'efficacité de l'arbre de recherche** : Les algorithmes de décision internes (tels que le *Failure-Directed Search*) exploitent plus difficilement la structure logique de la GCC, ce qui augmente le nombre de nœuds explorés avant d'identifier l'infaisabilité ou de prouver l'optimalité.

Cette formulation s'avère donc moins efficace en pratique (taux de résolution de 80.5 % contre 86.0 % pour le modèle classique sur le Subset 1).

2.6 Contraintes de bris de symétrie

Le modèle CP monolithique présente des symétries de permutation. Si plusieurs travailleurs possèdent les mêmes compétences, les solutions consistant à permuter ces travailleurs sur des tâches identiques sont équivalentes pour le Makespan, mais obligent le solveur à explorer des branches redondantes.

Pour limiter ces symétries, nous introduisons une contrainte forçant la conservation de l'affectation des travailleurs entre les unités de temps, sauf lorsqu'un événement (comme le début d'une nouvelle tâche) se produit. Les travailleurs n'ont de raisons de changer d'affectation que lorsqu'une nouvelle activité démarre et libère ou requiert de nouvelles compétences. Entre deux démarrages d'activités, l'affectation doit rester constante.

La contrainte s'exprime ainsi :

$$\text{IfThen}(PaS - \text{pulse}(par_{i,v}, 1) \leq -1, awo_{w,i,l,v} = awo_{w,i,l,v-1}) \quad \forall l \in \mathcal{L}, \forall w \in \mathcal{W}, \forall i \in \mathcal{A}, \forall v \in V_i \setminus \{0\} \quad (31)$$

Avec :

$$PaS = \sum_{i \in \mathcal{A}} \text{pulse}(par_{i,0}, 1) \quad (32)$$

Qui vaut zéro partout sauf lorsqu'au moins une tâche débute (c'est-à-dire lors de l'exécution de la première partie unitaire $par_{i,0}$ de n'importe quelle activité i).

Ainsi, si une partie unitaire $par_{i,v}$ ($v \geq 1$) est active alors qu'aucune tâche ne débute à cet instant ($PaS = 0$, donc $PaS - \text{pulse}(par_{i,v}, 1) = -1$), l'affectation des travailleurs pour la compétence l sur cette partie v est forcée d'être identique à celle de la partie précédente $v - 1$.

Bien que cette contrainte réduise théoriquement l'espace de recherche en éliminant les permutations équivalentes, l'évaluation expérimentale (présentée au Chapitre 4) montre qu'elle dégrade en réalité les performances globales du solveur CP Optimizer. Cela s'explique par deux facteurs :

- **Le coût de propagation :** L'ajout de ces contraintes logiques conditionnelles augmente la charge de travail du moteur de propagation à chaque nœud de l'arbre de recherche.
- **Perturbation des heuristiques de recherche :** Les solveurs modernes de CP les utilisent pour guider l'exploration. L'imposition rigide de contraintes de bris de symétrie peut entrer en conflit avec ces heuristiques en masquant des conflits importants pour l'élagage.

2.7 Analyse des limites du modèle CP monolithique

Bien que la formulation monolithique en CP offre un cadre de modélisation lisible grâce aux primitives de CP Optimizer (`interval`, `alternative`), sa mise en œuvre sur des instances de taille moyenne à grande se heurte à des limites de passage à l'échelle. Cette section en analyse les facteurs majeurs.

2.7.1 Explosion dimensionnelle du nombre de variables

La flexibilité d'affectation (*Allocation Flexibility*) autorise les travailleurs à se relayer au cours de l'exécution d'une même activité. Pour capturer ce comportement sans perdre la continuité temporelle de la tâche, le modèle fragmente chaque activité $i \in \mathcal{A}$ de durée p_i en p_i sous-activités unitaires de durée 1, notées $par_{i,v}$.

Cette discrétisation temporelle engendre de nombreuses variables :

1. **Variables d'intervalles unitaires :** Pour chaque tâche i , on génère p_i variables d'intervalles obligatoires $par_{i,v}$.
2. **Variables d'intervalles optionnelles d'affectation :** Pour chaque sous-activité unitaire $par_{i,v}$, pour chaque compétence l exigée et pour chaque travailleur w maîtrisant cette compétence ($m_{w,l} = 1$), on crée une variable d'intervalle optionnelle $awo_{w,i,l,v}$.

Le nombre de variables d'intervalles optionnelles à gérer s'exprime par la relation :

$$N_{opt} = \sum_{i \in \mathcal{A}} p_i \sum_{l \in \mathcal{L}} \text{sreq}_{i,l} \sum_{w \in \mathcal{W}} m_{w,l} \quad (33)$$

Pour une instance standard de taille moyenne (30 tâches, 5 compétences, 16 travailleurs polyvalents et des durées $p_i \approx 15$), N_{opt} dépasse 10 000 à 25 000 variables optionnelles. Cette dimensionnelle sature la mémoire vive et pèse sur le moteur de propagation de CP Optimizer, qui doit maintenir la cohérence de ces contraintes d'alternatives.

2.7.2 Perte de la structure globale d'ordonnancement et faiblesse de la propagation

La fragmentation des activités en intervalles unitaires réduit l'efficacité de la structure des tâches. Au lieu de manipuler des blocs temporels (ce pour quoi CP Optimizer est optimisé via des propagateurs spécifiques comme l'érosion temporelle ou le *edge-finding*), le solveur traite des micro-intervalles chaînés.

Cette structure présente deux faiblesses :

- **Faiblesse du filtrage des ressources :** Les contraintes cumulatives de capacité de ressources (exprimées par **pulse** ou **alwaysIn**) sont propagées localement sur chaque micro-intervalle de durée 1. Le solveur peine à projeter les conflits de ressources sur le long terme car il ne perçoit plus la tâche comme un bloc consommant des ressources sur toute sa durée.
- **Surcharge du réseau de contraintes de précédence :** Pour assurer la contiguïté temporelle de l'activité, le modèle impose des contraintes de séquençement strictes entre chaque partie unitaire successive. Ce chaînage crée un réseau de précédences dense qui ralentit la propagation des fenêtres de temps.

2.7.3 Symétries et inefficacité des contraintes de bris de symétrie

Le MS-RCPSP-AF souffre d'une forte symétrie de permutation des travailleurs. Si deux travailleurs possèdent le même profil de compétences, toute permutation de ces travailleurs produit des plannings équivalents pour le Makespan.

Dans le modèle CP monolithique, cette symétrie est accentuée par la discrétisation temporelle :
 * Le solveur explore des permutations de travailleurs d'une unité de temps à l'autre au sein d'une même tâche. * Par exemple, affecter un travailleur sur la première moitié d'une tâche et un autre sur la seconde moitié, ou vice-versa, est traité comme deux branches distinctes, alors que le makespan global reste inchangé.

L'ajout de contraintes de bris de symétrie vise à élaguer ces branches redondantes. Cependant, comme observé expérimentalement au chapitre 4, ces contraintes imposent un coût de filtrage trop lourd et masquent les heuristiques d'apprentissage de conflits internes du solveur (telles que le *Failure-Directed Search*), conduisant à des dégradations de performances et à des timeouts supplémentaires.

Ces limites mettent en évidence l'intérêt d'envisager des méthodes de décomposition comme la décomposition de Benders basée sur la logique (LBBD). En séparant l'ordonnancement temporel global (géré en temps continu) de l'affectation des ressources (déléguée à un sous-problème), la LBBD s'affranchit de la discrétisation temporelle et de l'explosion du nombre de variables associée.

3 Décomposition de Benders basée sur la Logique

Ce chapitre présente une méthode de décomposition exacte : la Décomposition de Benders basée sur la Logique (LBBD, pour *Logic-Based Benders Decomposition*). Cette approche sépare les décisions temporelles d’ordonnancement (dates d’exécution des tâches) des décisions spatiales d’affectation (attribution des travailleurs à chaque période).

La décomposition de Benders classique [26] partitionne les variables d’un problème en deux sous-ensembles résolus de manière itérative : un problème maître et un sous-problème. La version classique impose que le sous-problème soit un programme linéaire continu afin d’exploiter les variables duales pour construire des coupes d’optimalité ou de réalisabilité. La Décomposition de Benders basée sur la Logique (LBBD) [10, 11] étend ce principe aux cas où le sous-problème contient des variables entières ou des structures combinatoires discrètes (résolu par programmation par contraintes ou algorithmes de graphes). Les coupes ne proviennent plus de la dualité linéaire continue, mais de conditions logiques formulées à partir de la preuve d’infaisabilité ou d’optimalité du sous-problème.

La LBBD est fréquemment appliquée aux problèmes couplant ordonnancement et allocation de ressources [12]. Dans le cas du MSPSP, les premières approches utilisaient un schéma classique : un problème maître MILP pour l’affectation globale et un sous-problème CP pour l’ordonnancement temporel [17, 18, 27]. Pour améliorer la convergence, Yi-fan et al. [13] ont proposé d’inverser cette structure en confiant l’ordonnancement au maître CP et l’affectation au sous-problème. Récemment, Juvin et al. [1] ont étendu ce schéma inverse au problème avec flexibilité d’affectation (MS-RCPSP-AF).

Dans notre cas, la décomposition s’articule ainsi :

- **Le Problème Maître (CP) :** Il planifie les activités globales sur un horizon temporel continu en respectant les précédences et des relaxations globales de capacité de main-d’œuvre. Ces relaxations guident la recherche et limitent le nombre d’itérations [12].
- **Le Sous-Problème d’Affectation (Flot Max) :** Pour un ordonnancement fixé par le maître, il vérifie de manière indépendante pour chaque période t s’il existe une affectation valide des travailleurs pour couvrir les besoins en compétences des activités actives. En cas d’infaisabilité, les coupes logiques générées correspondent à des ensembles critiques ou interdits (*forbidden sets*) [28, 29].

3.1 Principes de la Décomposition de Benders basée sur la Logique (LBBD)

La Décomposition de Benders basée sur la Logique projette le problème sur les variables d’ordonnancement du problème maître, puis évalue la faisabilité de l’affectation dans le sous-problème sur les solutions d’ordonnancement générées.

3.2 Problème Maître d’Ordonnancement

Le Problème Maître est une version relaxée du modèle CP monolithique présenté au Chapitre 2. Il se concentre sur l’aspect temporel de l’ordonnancement global des tâches. Il ignore l’affectation nominative des opérateurs, mais intègre des contraintes de capacité cumulative globale (enveloppes de main-d’œuvre) pour éviter de générer des plannings manifestement irréalisables.

3.2.1 Formulation mathématique

Soit Ω l’ensemble des coupes de Benders générées et injectées dynamiquement au cours des itérations. Le Problème Maître s’exprime sous la forme du modèle CP suivant :

$$\min \quad C_{\max} \quad (34)$$

s. t.

$$C_{\max} \geq \text{act}_i.\text{end} \quad \forall i \in \mathcal{A} \quad (35)$$

$$\text{endBeforeStart}(\text{act}_i, \text{act}_m) \quad \forall (i, m) \in E \quad (36)$$

$$\sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, b_{i,k}) \leq B_k \quad \forall k \in \mathcal{CR} \quad (37)$$

$$\sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, q_i) \leq |\mathcal{W}| \quad (38)$$

$$\sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, a_{i,l}) \leq N_l \quad \forall l \in \mathcal{L} \quad (39)$$

$$\text{Contraintes de coupes de Benders (injections de } \Omega) \quad (40)$$

Les équations (35) à (39) correspondent aux contraintes d'ordonnancement et aux relaxations cumulatives (le besoin en effectif total ne doit pas dépasser le nombre d'opérateurs $|\mathcal{W}|$, et la demande cumulée par compétence ne doit pas dépasser le nombre d'opérateurs qualifiés N_l). La contrainte (40) représente les coupes cumulatives générées par les sous-problèmes d'affectation pour interdire les conflits de compétences détectés.

3.3 Sous-problème d'Affectation

À chaque itération, le solveur résout le Problème Maître et obtient un ordonnancement partiel où les dates de début et de fin de chaque activité globale act_i sont fixées. À partir de cet ordonnancement, nous déterminons pour chaque période $t \in \mathcal{T}$ l'ensemble des activités actives :

$$\mathcal{A}(t) = \{i \in \mathcal{A} \mid \text{act}_i.\text{start} \leq t < \text{act}_i.\text{end}\} \quad (41)$$

Pour chaque période t , la demande totale pour la compétence l , notée $D_l(t)$, est la somme des besoins des tâches actives à cet instant :

$$D_l(t) = \sum_{i \in \mathcal{A}(t)} a_{i,l} \quad \forall l \in \mathcal{L} \quad (42)$$

La validation de la faisabilité spatiale consiste à vérifier s'il existe une affectation binaire des travailleurs $w \in \mathcal{W}$ aux compétences $l \in \mathcal{L}$ telle que la demande $D_l(t)$ soit satisfaite pour toute compétence et que chaque travailleur ne soit affecté qu'au plus à une seule compétence.

Pour modéliser ce sous-problème d'affectation à chaque période t , deux approches sont envisageables. La première consiste à formuler un **programme linéaire continu (PL)** de décision de faisabilité en relâchant les contraintes d'intégrité binaire sur les variables d'affectation. Soit $x_{w,l} \geq 0$ la variable de décision continue représentant la fraction de temps que le travailleur w consacre à la compétence l durant la période t . Le problème se formule ainsi :

$$\min \quad 0 \quad (43)$$

$$\text{s. t.} \quad \sum_{l \in \mathcal{L}} x_{w,l} \leq 1 \quad \forall w \in \mathcal{W} \quad (44)$$

$$\sum_{w \in \mathcal{W}} x_{w,l} \cdot m_{w,l} = D_l(t) \quad \forall l \in \mathcal{L} \quad (45)$$

$$x_{w,l} \geq 0 \quad \forall w \in \mathcal{W}, \forall l \in \mathcal{L} \quad (46)$$

La contrainte (44) garantit qu'un travailleur n'est pas affecté au-delà de sa capacité temporelle

unitaire, tandis que la contrainte (45) assure la satisfaction exacte de la demande en compétences requise. L'objectif est simplement de trouver une solution admissible (coût nul).

Pour éviter d'appeler un solveur de programmation linéaire générique (simplexe ou points intérieurs) à chaque étape de la recherche, nous utilisons plutôt la seconde approche : la modélisation sous forme de flot maximum dans un réseau bipartite. Ce formalisme permet d'appliquer des algorithmes de graphes spécialisés plus rapides.

3.3.1 Formulation sous forme de Flot Maximum (Max-Flow)

Puisque les périodes sont indépendantes une fois le temps fixé, le problème d'affectation pour une période t donnée est représenté comme un problème de Flot Maximum dans un réseau de transport.

Nous construisons un graphe orienté $G(V, A)$ composé de :

- Une source s et un puits p .
- Un nœud pour chaque compétence $l \in \mathcal{L}$.
- Un nœud pour chaque travailleur $w \in \mathcal{W}$.

Les arcs et leurs capacités associées sont définis comme suit :

- Un arc orienté de la source s vers chaque nœud de compétence l avec une capacité égale à la demande $D_l(t)$.
- Un arc orienté de chaque nœud de compétence l vers chaque nœud de travailleur w si et seulement si le travailleur maîtrise cette compétence ($m_{w,l} = 1$). La capacité de cet arc est infinie.
- Un arc orienté de chaque nœud de travailleur w vers le puits p avec une capacité unitaire (1), pour garantir qu'un travailleur ne peut être affecté qu'à une seule tâche/compétence.

Le problème de flot maximum se formule ainsi :

$$\max \sum_{l \in \mathcal{L}} f_{s,l} \quad (47)$$

$$\text{s.t. } f_{s,l} \leq D_l(t) \quad \forall l \in \mathcal{L} \quad (48)$$

$$f_{s,l} - \sum_{w \in \mathcal{W}} f_{l,w} \cdot m_{w,l} = 0 \quad \forall l \in \mathcal{L} \quad (49)$$

$$\sum_{l \in \mathcal{L}} f_{l,w} - f_{w,p} = 0 \quad \forall w \in \mathcal{W} \quad (50)$$

$$f_{w,p} \leq 1 \quad \forall w \in \mathcal{W} \quad (51)$$

$$f \geq 0 \quad (52)$$

Si la valeur du flot maximum optimal est strictement inférieure à la demande totale $\sum_{l \in \mathcal{L}} D_l(t)$, alors l'affectation est irréalisable à l'instant t . Le solveur doit alors générer une coupe de Benders pour exclure cet ordonnancement temporel.

3.4 Génération des coupes de Benders

Lorsqu'une infaisabilité est détectée à une période t , nous devons identifier la cause de cette infaisabilité sous la forme d'un conflit de compétences, puis exprimer ce conflit sous forme de contrainte globale (coupe) à réinjecter dans le Problème Maître CP.

3.4.1 Formulation duale et Coupe Minimale (Min-Cut)

Pour identifier le sous-ensemble minimal de compétences à l'origine du conflit, nous exploitons la dualité linéaire. Le problème dual associé au problème d'affectation (ou le théorème du flot maximum/coupe minimale) permet d'identifier une coupe minimale dans le réseau.

Le modèle dual se formule ainsi :

$$\min \sum_{w \in \mathcal{W}} d_{w,p} + \sum_{l \in \mathcal{L}} D_l(t) \cdot d_{s,l} \quad (53)$$

$$\text{s.t. } d_{s,l} + p_l \geq 1 \quad \forall l \in \mathcal{L} \quad (54)$$

$$d_{w,p} + p_w \geq 0 \quad \forall w \in \mathcal{W} \quad (55)$$

$$p_w - m_{w,l} \cdot p_l \geq 0 \quad \forall l \in \mathcal{L}, \forall w \in \mathcal{W} \quad (56)$$

Soit $S \subseteq \mathcal{L}$ le sous-ensemble de compétences situées du côté source de la coupe minimale dans la solution duale optimale (c'est-à-dire les compétences pour lesquelles $d_{s,l} = 0$). Soit W_S le sous-ensemble de travailleurs connectés à ces compétences du côté source de la coupe ($d_{w,p} = 1$). D'après le théorème de la coupe minimale, si l'affectation est infaisable, la capacité de la coupe minimale est strictement inférieure à la demande totale :

$$|W_S| < \sum_{l \in S} D_l(t) \quad (57)$$

Cela signifie physiquement que le nombre de travailleurs disponibles maîtrisant au moins une compétence de S (noté $|W_S|$) est insuffisant pour couvrir la demande cumulée en compétences de S demandée par les tâches actives à l'instant t .

3.4.2 Formulation de la coupe injectée

Pour interdire cette situation dans le Problème Maître CP, nous injectons une contrainte cumulative sous forme d'une fonction de pulse :

$$\sum_{i \in \mathcal{A}} \sum_{l \in S} \text{pulse}(act_i, a_{i,l}) \leq |W_S| \quad (58)$$

Cette coupe indique que la somme des demandes en compétences appartenant à S pour toutes les activités s'exécutant simultanément ne doit pas dépasser le nombre de travailleurs qualifiés $|W_S|$. Cette contrainte est formulée sous forme de fonctions cumulatives et de propagateurs temporels, ce qui permet au Problème Maître de décaler dans le temps certaines activités en conflit lors de la recherche.

4 Évaluation Expérimentale

Dans ce chapitre, nous présentons l'évaluation expérimentale de l'ensemble des approches modélisées : les modèles de programmation par contraintes monolithiques classiques (avec ou sans contraintes de bris de symétrie), les formulations basées sur la contrainte de cardinalité globale (avec ou sans bris de symétrie), ainsi que le framework de décomposition de Benders basée sur la logique (LBBD). L'objectif est d'analyser leurs performances en termes de temps de résolution, de passage à l'échelle, et de qualité de convergence face à la flexibilité d'affectation.

4.1 Protocole expérimental et instances de test

4.1.1 Environnement de test

Les expérimentations ont été menées sur une machine équipée d'un processeur Intel Xeon W6-3423 cadencé à 2.1 GHz et de 62 Go de mémoire RAM, sous Ubuntu Linux. Les solveurs utilisés sont :

- **IBM ILOG CP Optimizer** (version 22.1) pour la résolution du modèle CP monolithique et du Problème Maître CP dans la décomposition.
- L'implémentation de l'algorithme de flot maximum d'Edmonds-Karp de **NetworkX** pour résoudre le sous-problème d'affectation à chaque période.

Pour chaque instance, la limite de temps de calcul a été fixée à **500 secondes**.

4.1.2 Jeu de données initial : Set 1a

L'évaluation repose sur un ensemble de **215 instances** de référence issues de la bibliothèque publique **MSPSP-InstLib**¹ introduite par Young et al. [8]. Ce jeu, nommé « Set 1a », a été conçu à l'aide d'un générateur dérivé des travaux d'Almeida et al. [5] et converti au format DataZinc (.dzn). Ces instances se caractérisent par :

- Un nombre d'activités $n = 20$.
- Un effectif de travailleurs $m \in \{10, 13, 15\}$.
- Un facteur de complexité réseau ($nc \in \{1.5, 1.8, 2.1\}$), qui régit la densité du graphe.
- Un facteur de compétences ($sf \in \{0, 0.5, 0.75, 1.0\}$), caractérisant la diversité et le degré de maîtrise des compétences ($sf = 0$ correspond à un facteur de compétences variable et aléatoire).

Ce jeu d'instances offre une diversité de configurations de contraintes et de ressources pour l'analyse comparative.

4.1.3 Jeu de données de passage à l'échelle : MSLIB

Bien que le jeu d'instances « Set 1a » permette d'obtenir des comparaisons utiles et de valider les modèles, sa taille ($n = 20$ tâches) limite sa portée pratique pour représenter des problématiques industrielles réelles.

Pour évaluer la robustesse et les limites de passage à l'échelle des approches, nous avons choisi d'utiliser un jeu de données de taille supérieure : la bibliothèque **MSLIB**² introduite par Snauwaert et Vanhoucke [4].

¹<https://github.com/youngkd/MSPSP-InstLib>

²<https://github.com/MarioVanhoucke/MSLIB-Multi-Skilled-Resource-Constrained-Project-Library>

Cette bibliothèque a été conçue par l'équipe du professeur Mario Vanhoucke à l'Université de Gand. MSLIB se structure en 5 grands jeux de données : les jeux MSLIB1 à MSLIB4 (artificiels) et le jeu MSLIB5 (regroupant 18 cas réels). Nous utilisons le jeu MSLIB1, qui comporte **6 600 instances** de taille supérieure :

- Un nombre d'activités fixe $n = 32$ (30 tâches et 2 tâches fictives).
- Un nombre de compétences fixe $|L| = 4$.
- Un effectif de travailleurs m variable, allant de 4 à 312.

Les instances sont stockées au format de Patterson (**.msrcp**). Pour les ramener au formalisme du MS-RCPSP-AF, nous appliquons les règles de projection suivantes :

- **Project Module** : définit la structure du graphe de précedence, les durées de chaque tâche et leurs successeurs. Ces données sont conservées.
- **Workforce Module** : décrit la maîtrise binaire des compétences par les travailleurs, ce qui alimente directement notre matrice **mastery**.
- **Skill Requirements Module** : indique pour chaque tâche le nombre de travailleurs requis par compétence, ce qui définit notre matrice **sreq**.
- Les autres modules (niveaux de compétence, coûts et retouches) sont ignorés pour correspondre à notre version du problème.

Construction des sous-ensembles (Subsets) : Résoudre les 6 600 instances pour chaque solveur représenterait un coût de calcul important. Pour obtenir un échantillon calculable, nous avons partitionné le jeu en **33 sous-ensembles représentatifs de 200 instances**.

Cette partition a été construite par un découpage par round-robin stratifié : 1. Les 6 600 instances ont été triées par ordre croissant du nombre de travailleurs m . 2. Les instances ont ensuite été distribuées cycliquement dans les 33 sous-ensembles.

Ce processus permet d'assurer que chaque sous-ensemble présente une distribution similaire des effectifs de ressources (médiane $m \approx 49$, écart-type ≈ 54). Pour nos expérimentations, nous utilisons le **Subset 1** (200 instances).

4.2 Comparaison globale des approches sur le Set 1a

Les résultats obtenus sur les 215 instances du **Set 1a** pour les différentes méthodes de résolution (CP Classique, CP avec bris de symétrie, CP Distribute, CP Distribute + Symétrie et la Décomposition de Benders) sont résumés dans le tableau 1.

Table 1: Synthèse globale des résultats de résolution sur le Set 1a (Time Limit = 500s)

Approche	Inst. résolues	Succès (%)	Temps moy. (s)	Temps hors TO (s)	Gap moy. (%)
CP Classique (Monolithique)	169 / 215	78.6 %	151.31	56.38	2.90 %
CP avec bris de symétrie	164 / 215	76.3 %	159.39	53.46	8.11 %
CP Distribute	157 / 215	73.0 %	152.86	24.60	4.20 %
CP Distribute + Symétrie	152 / 215	70.7 %	165.94	27.46	7.73 %
Benders (LBBD)	209 / 215	97.2 %	24.70	11.05	0.00 %

4.2.1 Analyse des taux de résolution et des temps de calcul

La Décomposition de Benders basée sur la Logique (LBBD) obtient de meilleurs résultats que les approches monolithiques. Elle résout **97.2 %** des instances, ne subissant que 6 timeouts.

Le modèle CP classique échoue à prouver l’optimalité sur 46 instances (21.4 % de timeouts) et le modèle CP Distribute sur 58 instances (27.0 % de timeouts). Le passage au modèle GCC (CP Distribute) réduit le nombre de variables d’intervalles mais diminue l’efficacité de la propagation, ce qui se traduit par un taux de résolution plus faible (73.0 %). De plus, comme observé pour le modèle classique et GCC, l’ajout de contraintes de bris de symétrie augmente les temps de calcul et réduit les performances.

La décomposition de Benders résout les instances en un temps moyen de **24.70 secondes** (dont un temps moyen de **11.05 secondes** pour les instances résolues avant la limite).

4.2.2 Comparaison directe sur les instances faciles (communes)

Pour comparer les approches sans le biais introduit par les timeouts, nous avons isolé les **155 instances résolues à l’optimalité par toutes les méthodes** :

- **Temps moyen Benders (LBBD) : 4.55 s**
- **Temps moyen CP avec bris de symétrie : 49.48 s**
- **Temps moyen CP Classique : 47.00 s**

Sur ces instances communes, la décomposition de Benders se montre plus rapide que le CP classique.

4.3 Analyse des performances et discussion

4.3.1 Le temps de convergence de Benders

Sur plusieurs instances où les modèles CP monolithiques atteignent la limite de 500 secondes, le framework LBBD converge vers l’optimalité rapidement. Par exemple, pour l’instance `inst_set1a_sf0.5_nc1.5_n20_m13_01.dzn`, le modèle CP classique expire après 500 secondes avec un gap de 5.0 %, tandis que la décomposition de Benders trouve la solution optimale en **0.18 seconde** et **3 itérations**.

Ce comportement s’explique par les coupes injectées. L’analyse du flot maximum identifie l’ensemble critique de compétences et de travailleurs qui cause l’infaisabilité. La coupe cumulative réinjectée dans le Problème Maître CP élimine des configurations de plannings irréalisables, ce qui limite le nombre d’itérations nécessaires.

4.3.2 Analyse de l’impact des contraintes de bris de symétrie

La contrainte de bris de symétrie dégrade les performances par rapport au modèle classique (164 résolutions contre 169). L’expression de ces contraintes introduit un coût de filtrage à chaque nœud qui n’est pas compensé par la réduction de la taille de l’arbre. De plus, elle peut gêner les heuristiques de recherche dynamique internes de CP Optimizer (comme le *Failure-Directed Search*).

4.4 Visualisations graphiques pour le Set 1a

La figure 2 illustre le profil de performance des 5 méthodes sur le **Set 1a**.

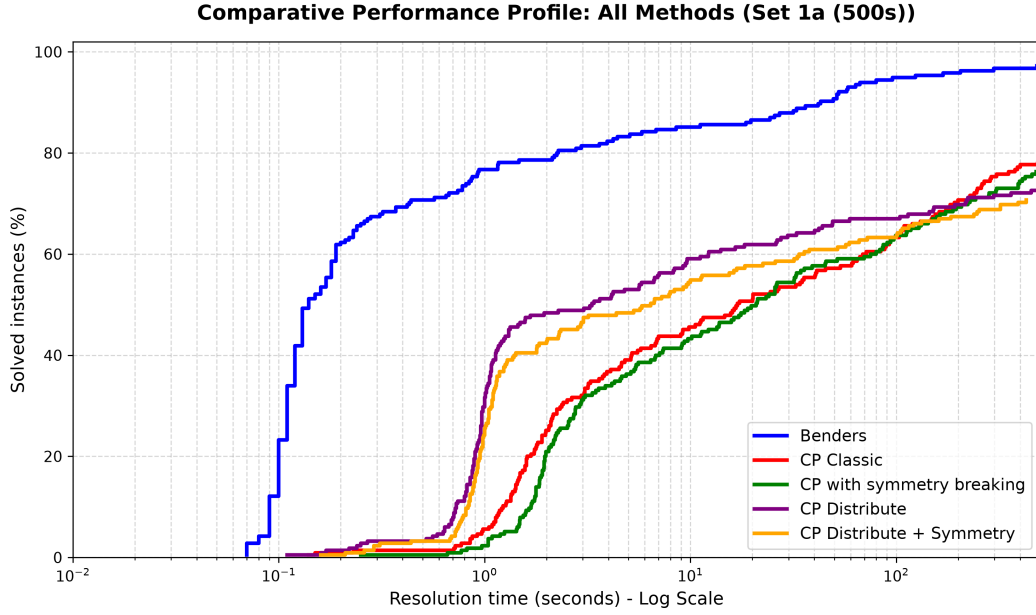


Figure 2: Profil de performance cumulatif unifié : Comparaison du taux de résolution (%) en fonction du temps de calcul (échelle logarithmique) sur le Set 1a (215 instances - 500s).

4.5 Résultats sur le jeu de données de passage à l'échelle MSLIB1 (Subset 1)

Les performances de la décomposition de Benders (LBBD) et des trois modèles CP monolithiques (CP Classique, CP Distribute, et CP Distribute avec bris de symétrie) ont été évaluées sur le sous-ensemble de 200 instances **Subset 1** de MSLIB1 avec une limite de 500 secondes. Le tableau 2 présente les résultats globaux.

Table 2: Synthèse globale des résultats de résolution sur MSLIB1 (Subset 1, 200 instances, Time Limit = 500s)

Approche	Inst. résolues	Succès (%)	Temps moy. (s)	Temps hors TO (s)	Gap moy. (%)
CP Classique (Monolithique)	172 / 200	86.0 %	78.64	10.02	2.53 %
CP Distribute	161 / 200	80.5 %	105.72	10.19	7.28 %
CP Distribute + Symétrie	162 / 200	81.0 %	102.27	8.96	8.01 %
Benders (LBBD)	188 / 200	94.0 %	32.62	2.78	0.00 %

4.5.1 Analyse des performances sur MSLIB1

Les résultats confirment l'efficacité de la décomposition de Benders (LBBD) sur ce jeu étendu, résolvant **94.0 %** des instances avec un temps moyen de **2.78 secondes** hors timeouts.

Pour les instances à ressources critiques ($m \leq 20$), CP rencontre des difficultés dues au nombre important de variables d'intervalles optionnelles ($30 \times 4 \times 16 = 1920$ variables optionnelles en moyenne), ce qui ralentit la propagation. Benders évite cette surcharge en planifiant des variables d'intervalles simples dans le problème maître et en confiant l'affectation au sous-problème.

L'apport de la contrainte de bris de symétrie sur CP Distribute est positif sur ce jeu de données : elle réduit le temps moyen de résolution hors timeouts de **10.19s à 8.96s** et permet de résoudre une instance supplémentaire.

La figure 3 illustre le profil de performance sur ce sous-ensemble.

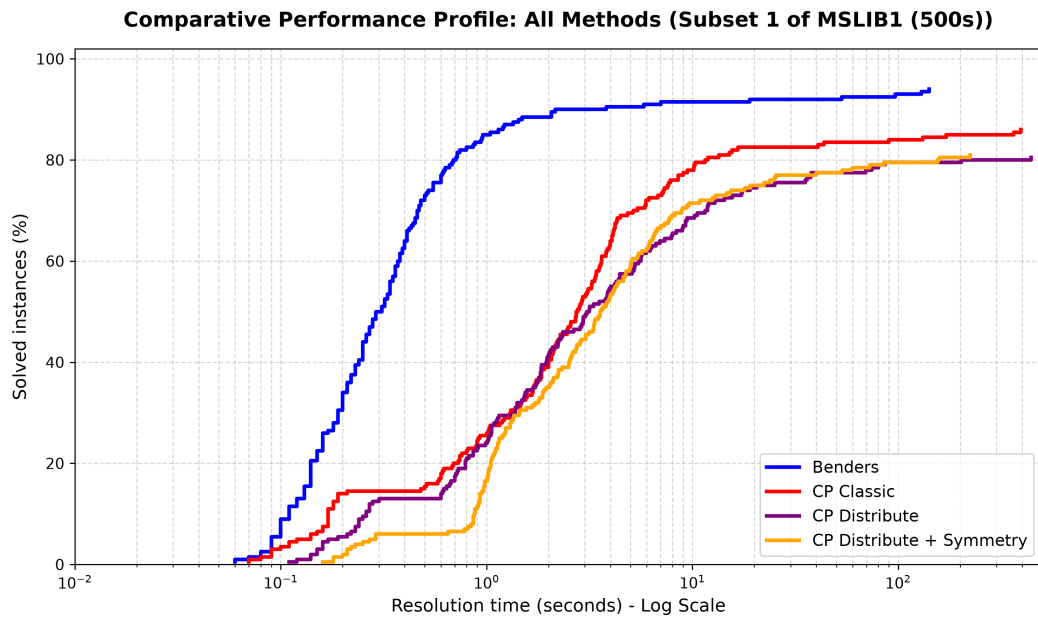


Figure 3: Profil de performance cumulatif unifié : Comparaison du taux de résolution (%) en fonction du temps de calcul (échelle logarithmique) sur MSLIB1 (Subset 1 - 200 instances - 500s).

Conclusion et Perspectives

Ce travail de stage a permis de concevoir, d'implémenter et de comparer différentes approches de résolution exactes pour le problème d'ordonnancement de projet sous contraintes de ressources multi-compétences avec flexibilité d'affectation (MS-RCPSP-AF). Nous avons formulé un modèle mathématique initial en PLNE, puis un modèle monolithique en Programmation par Contraintes (CP) exploitant des variables d'intervalles unitaires pour capturer la flexibilité d'affectation dynamique. Face aux limites de ces approches monolithiques (explosion du nombre de variables optionnelles et affaiblissement de la propagation), nous avons développé un algorithme de décomposition de Benders basé sur la logique (LBBD) couplant un problème maître en CP et des sous-problèmes d'affectation résolus sous forme de flot maximum. Les résultats expérimentaux sur les instances de référence (Set 1a et MSLIB1) confirment l'efficacité de la décomposition LBBD, qui surpasse les modèles monolithiques en résolvant 94

References

- [1] C. Juvin, C. Artigues, P. Lopez, and R. Brux, “Logic-based benders method for multi-skill resource-constrained project scheduling with allocation and skill flexibility,” *Preprint submitted to Elsevier*, 2026.
- [2] O. Bellenguez-Morineau and E. Néron, “A branch-and-bound method for solving multi-skill project scheduling problem,” *RAIRO-Operations Research*, vol. 41, no. 2, pp. 155–170, 2007.
- [3] ———, “Multi-mode and multi-skill project scheduling problem,” in *Resource-Constrained Project Scheduling: Models, Algorithms, Extensions and Applications*, C. Artigues, S. Demassey, and E. Néron, Eds. Wiley Online Library, 2008, pp. 149–160.
- [4] J. Snauwaert and M. Vanhoucke, “A classification and new benchmark instances for the multi-skilled resource-constrained project scheduling problem,” *European Journal of Operational Research*, vol. 307, no. 1, pp. 1–19, 2023.
- [5] B. F. Almeida, I. Correia, and F. Saldanha-da-Gama, “Modeling frameworks for the multi-skill resource-constrained project scheduling problem: A theoretical and empirical comparison,” *International Transactions in Operational Research*, vol. 26, no. 3, pp. 946–967, 2019.
- [6] I. Correia and F. Saldanha-da-Gama, “A modeling framework for project staffing and scheduling problems,” in *Handbook on Project Management and Scheduling Vol. 1*, C. Schwindt and J. Zimmermann, Eds. Springer, 2015, pp. 547–564.
- [7] C. Montoya, O. Bellenguez-Morineau, E. Pinson, and D. Rivreau, “Branch-and-price approach for the multi-skill project scheduling problem,” *Optimization Letters*, vol. 8, pp. 1721–1734, 2014.
- [8] K. D. Young, T. Feydy, and A. Schutt, “Constraint programming applied to the multi-skill project scheduling problem,” in *Principles and Practice of Constraint Programming: 23rd International Conference (CP 2017), Melbourne, VIC, Australia, August 28–September 1, 2017, Proceedings 23*. Springer, 2017, pp. 308–317.
- [9] O. Polo-Mejía, C. Artigues, P. Lopez, and V. Basini, “Mixed-integer/linear and constraint programming approaches for activity scheduling in a nuclear research facility,” *International Journal of Production Research*, vol. 58, no. 23, pp. 7149–7166, 2020.
- [10] J. N. Hooker, *Logic-based methods for optimization: Combining optimization and constraint satisfaction*. New York: John Wiley and Sons, 2000.
- [11] ———, “Logic-based benders decomposition for large-scale optimization,” *Large scale optimization in supply chains and smart manufacturing: Theory and applications*, pp. 1–26, 2019.
- [12] A. A. Cire, E. Coban, and J. N. Hooker, “Logic-based benders decomposition for planning and scheduling: A computational analysis,” *The Knowledge Engineering Review*, vol. 31, no. 5, pp. 440–451, 2016.
- [13] W. Yi-fan, S. Fu-quan, L. Shi-xin, and C. Di, “A new method to solve project scheduling problems with multi-skilled workforce constraints,” in *2013 25th Chinese Control and Decision Conference (CCDC)*. IEEE, 2013, pp. 2408–2412.
- [14] R. M. Karp, *Reducibility among combinatorial problems*. Springer, 2010.
- [15] C. H. Papadimitriou and K. Steiglitz, *Combinatorial optimization: Algorithms and complexity*. Courier Corporation, 1998.

- [16] E. L. Demeulemeester and W. S. Herroelen, *Project scheduling: A research handbook*. Springer Science & Business Media, 2006, vol. 49.
- [17] H. Li and K. Womer, “Scheduling projects with multi-skilled personnel by a hybrid MILP/CP Benders decomposition algorithm,” *Journal of Scheduling*, vol. 12, no. 3, pp. 281–298, 2009.
- [18] H. Li, “Benders decomposition approach for project scheduling with multi-purpose resources,” in *Handbook on Project Management and Scheduling Vol. 1*, C. Schwindt and J. Zimmermann, Eds. Springer, 2015, pp. 587–601.
- [19] O. Polo-Mejía, C. Artigues, P. Lopez, L. Mönch, and V. Basini, “Heuristic and metaheuristic methods for the multi-skill project scheduling problem with partial preemption,” *International Transactions in Operational Research*, vol. 30, no. 2, pp. 858–891, 2023.
- [20] O. Polo Mejía, “Operational research approach for optimising the operations of a nuclear research laboratory,” Ph.D. dissertation, Institut National des Sciences Appliquées de Toulouse, Sep 2019. [Online]. Available: <https://hal.science/tel-02309284>
- [21] H. Fahimi and C.-G. Quimper, “Linear-time filtering algorithms for the disjunctive constraint,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28(1), 2014.
- [22] P. Vilím, “Edge finding filtering algorithm for discrete cumulative resources in $\mathcal{O}(kn \log n)$,” in *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 2009, pp. 802–816.
- [23] Y. Ouellet and C.-G. Quimper, “A $O(n \log^2 n)$ checker and $O(n^2 \log n)$ filtering algorithm for the energetic reasoning,” in *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 2018, pp. 477–494.
- [24] J. Carlier and E. Néron, “Computing redundant resources for the resource constrained project scheduling problem,” *European Journal of Operational Research*, vol. 176, no. 3, pp. 1452–1463, 2007.
- [25] P. Baptiste and N. Bonifas, “Redundant cumulative constraints to compute preemptive bounds,” *Discrete Applied Mathematics*, vol. 234, pp. 168–177, 2018.
- [26] J. Benders, “Partitioning procedures for solving mixed-variables programming problems,” *Numerische Mathematik*, vol. 4, no. 1, pp. 238–252, 1962.
- [27] A. Nasirian, L. Zhang, A. M. Costa, and B. Abbasi, “Multiskilled workforce staffing and scheduling: A logic-based Benders’ decomposition approach,” *European Journal of Operational Research*, vol. 323, no. 1, pp. 20–33, 2025.
- [28] M. Lombardi and M. Milano, “A min-flow algorithm for minimal critical set detection in resource constrained project scheduling,” *Artificial Intelligence*, vol. 182, pp. 58–67, 2012.
- [29] F. Stork and M. Uetz, “On the generation of circuits and minimal forbidden sets,” *Mathematical programming*, vol. 102, pp. 185–203, 2005.